

Wykresy opóźnień na wyspach w zależności od strategii

Poniżej przedstawiam przykładowe wykresy generowane na podstawie lokalnego uruchomienia algorytmu na 7 wyspach. Na razie bez żadnych zmian heurystyki:

Metryka użyta na wykresach

Wykresy pokazują opóźnienie migracji liczone w krokach algorytmu:

$$\text{delay_steps} = \text{source_iteration} - \text{destination_step}$$

gdzie:

- **source_iteration** to numer iteracji na wyspie źródłowej w chwili wysłania migranta,
- **destination_step** to numer kroku na wyspie docelowej w chwili jego odebrania.

Interpretacja:

- **delay_steps < 0** oznacza migranta opóźnionego,
- **delay_steps = 0** oznacza migranta zsynchronizowanego,
- **delay_steps > 0** oznacza migranta przyspieszonego.

Każdy panel na wykresie odpowiada jednej wyspie docelowej i pokazuje wartości **delay_steps** kolejnych odebranych migrantów w funkcji kroku wyspy docelowej.

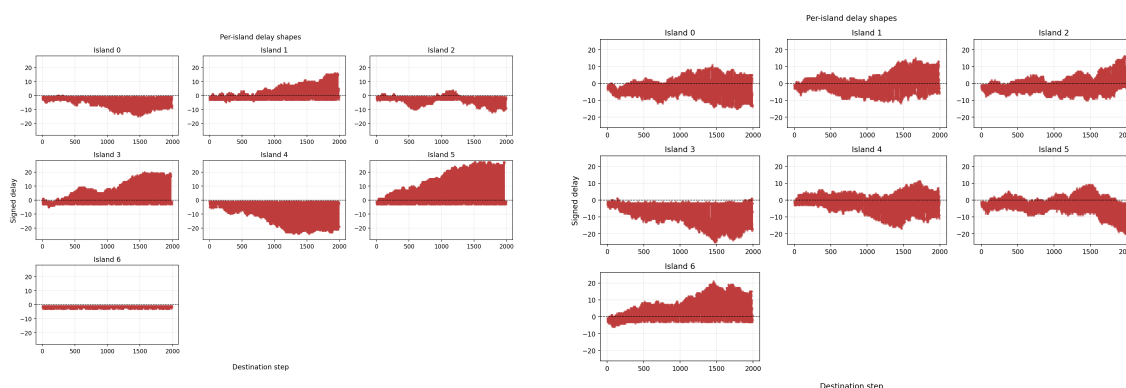
Wykresy opóźnień dla różnych strategii

Strategia

ring

complete

random -
plain



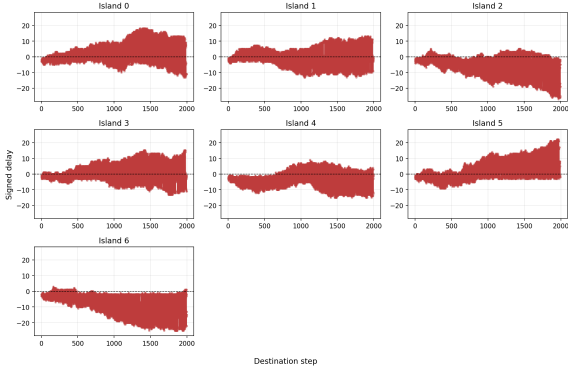
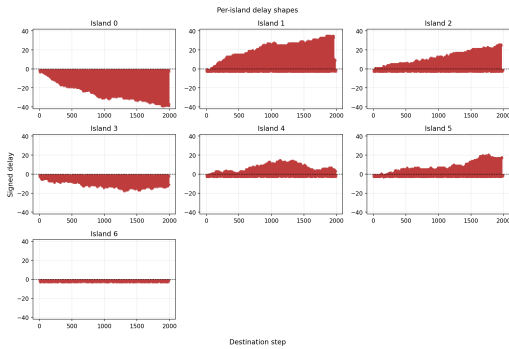
Strategia

ring

complete

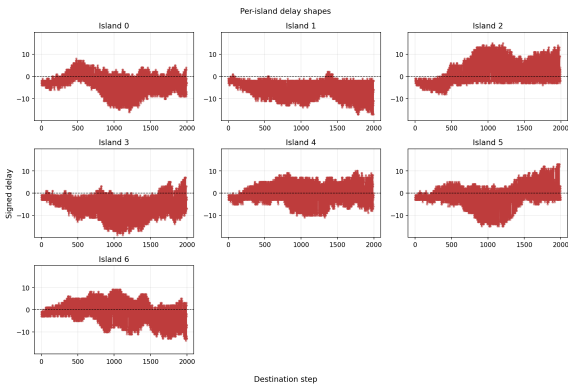
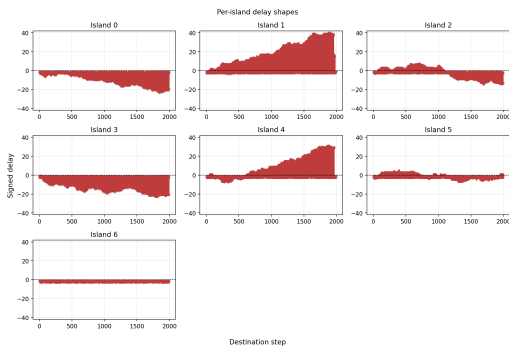
best -

plain



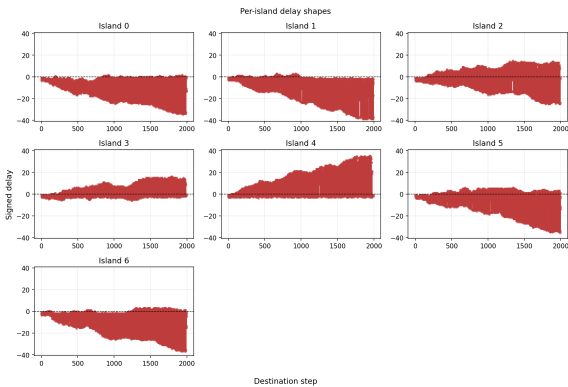
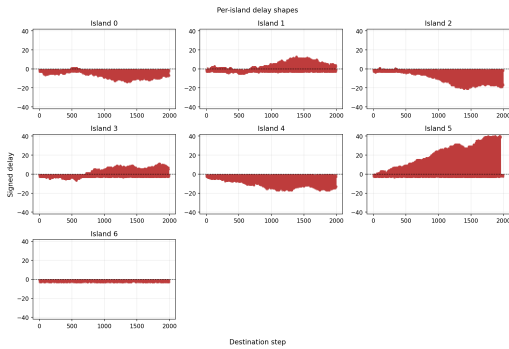
random -

SAS



best -

SAS



Zestawienie liczbowe

Strategia	Topologia	Mean delay	Delayed fraction	Final best fitness	Final average fitness
random - plain	ring	-0.623	75.189%	15.072402	27.411450
random - plain	complete	-2.012	65.587%	5.680626	5.704166
best - plain	ring	0.146	66.568%	16.886619	29.567628
best - plain	complete	-2.385	67.921%	4.098922	4.139701

Strategia	Topologia	Mean delay	Delayed fraction	Final best fitness	Final average fitness
random - SAS	ring	-0.559	76.544%	21.815868	33.466030
random - SAS	complete	-2.163	69.296%	5.572004	5.620944
best - SAS	ring	-0.957	78.584%	16.007113	26.402485
best - SAS	complete	-2.194	66.366%	3.042633	3.067133

Obserwacje

W większości przypadków obserwujemy ujemne średnie opóźnienie, co oznacza, że migranci są częściej opóźnieni niż przyspieszeni. Jest to zgodne z intuicją, bo komunikacja i przetwarzanie migranta zajmuje czas, więc często dociera on do wyspy docelowej po iteracji, w której został wysłany. Jednak amplituda średniego opóźnienia jest znacznie większa dla topologii **complete**, podczas gdy dla **ring** jest znacznie mniejsza.

Z punktu widzenia jakości optymalizacji w tych lokalnych biegach topologia **complete** była wyraźnie lepsza od **ring**. Najlepszy końcowy fitness dla **ring** mieścił się w zakresie 15.072 do 21.816, podczas gdy dla **complete** spadł do zakresu 3.043 do 5.681. W tej próbie najlepszy wynik całkowity uzyskano dla **complete + best + SAS**, mimo że ta konfiguracja nie minimalizowała średniego opóźnienia.

Jest to spójne z opisem z artykułu **Delays in computing with Parallel metaheuristics on HPC infrastructure**: topologia **complete** może generować silniejsze zatłoczenie komunikacji i większą złożoność wzorców opóźnień, ale jednocześnie daje większą różnorodność migrantów, co może poprawiać wynik optymalizacji. **Ring** jest bardziej uporządkowany komunikacyjnie, ale ogranicza dopływ zróżnicowanych migrantów, więc mniejsze lub bardziej lokalne opóźnienia nie muszą przekładać się na lepszy fitness.

Potencjalne strategie przyjmowania migrantów oparte na opóźnieniach

Najprostsza strategia przyjmowania migrantów to przyjmowanie tylko migrantów nowszych niż obecna iteracja wyspy docelowej.

```

delay_steps = source_iteration - destination_step

if delay_steps > 0:
    accept migrant
else:
    reject migrant

```

Taki wariant nazwiemy **onlyNewer**. Odrzuca on migrantów opóźnionych oraz zsynchronizowanych, a zostawia tylko tych, których źródło było ewolucyjnie dalej niż wyspa docelowa w chwili wysłania. To jest

bardzo agresywna polityka, bo w tych danych udział migrantów przyspieszonych wynosi tylko około 20–30%.

Pomysły na alternatywne strategie:

- `plain`: punkt odniesienia bez filtrowania po opóźnieniu.
- `onlyNewer`: przyjmuj tylko migrantów z `delay_steps > 0`.
- `newerOrAligned`: przyjmuj migrantów z `delay_steps >= 0`
- `rejectTooOldK`: odrzucaj tylko migrantów silnie opóźnionych, np. `delay_steps < -10`.
- `delayWindow`: przyjmuj migrantów z okna `-K <= delay_steps <= A`, np. `-10 <= delay_steps <= 20`.
- `fitnessAndDelay`: przyjmuj migranta, jeśli jest nowszy od wyspy docelowej albo jeśli poprawia lokalny najlepszy fitness.
- `SASDelay`: zachowuje ideę `SAS`, ale dodaje warunek opóźnienia, np. nie przyjmuj migrantów z dobrych historycznie źródeł, jeśli `delay_steps < -K`.